WUMPUS 3D · FUNDAMENTAÇÃO TEÓRICA

Agente genético

De onde vem cada parte do agente que aprende com algoritmo genético, o que cada variável significa e o que foi escolha de projeto sem respaldo teórico.

Versão do código	**4e9f8c5**
Data	**29 de outubro de 2026**
Arquivos	**src/agents/genome.ts · genetic.ts · agents.ts (GeneticAgent) · knowledge.ts**
Referências	**13 obras, todas conferidas (ver apêndice A)**

Sumário

1 Resposta direta

2 Como o agente funciona

3 Fundamentação de cada componente

4 Cada variável do agente

5 Resultados obtidos e o que eles significam

6 Limitações da fundamentação

7 Referências

A Apêndice: o que não foi possível verificar

1 Resposta direta

O agente genético do Wumpus 3D não foi construído a partir de um artigo, tese ou método publicado específico. Ele foi projetado durante o desenvolvimento (por mim, o assistente que escreveu o código) combinando técnicas clássicas e bem documentadas. Este documento mostra de onde vem cada técnica e separa com clareza o que tem fundamentação na literatura do que foi uma escolha de projeto sem respaldo teórico específico.

O que tem base na literatura

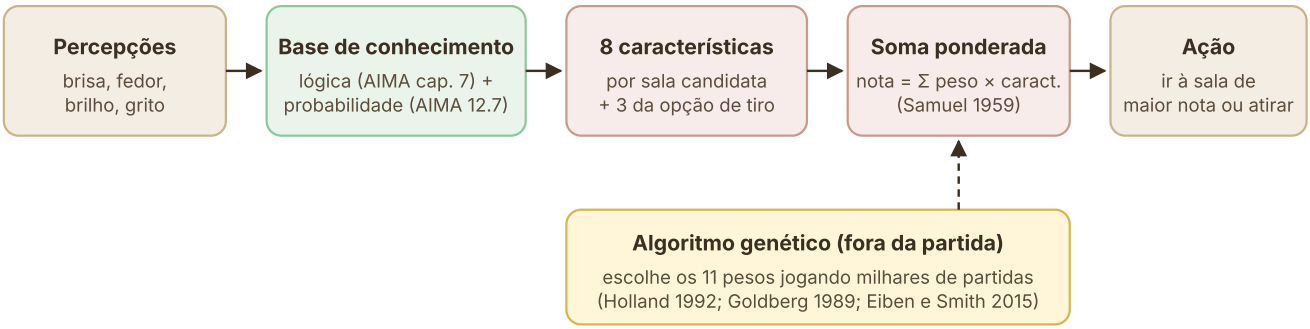
- O problema e a forma de raciocinar sobre ele (deduções lógicas e probabilidade de poço): o mundo de Wumpus do livro AIMA [Russell e Norvig (2020)].
- A ideia de decidir com uma **soma ponderada de características** cujos pesos são aprendidos: a função de avaliação de Samuel para damas [Samuel (1959)]; a evolução de funções de avaliação de jogos por computação evolutiva aparece em [Chellapilla e Fogel (1999)].
- O **algoritmo genético** em si e seus operadores (população, seleção por torneio, elitismo, cruzamento aritmético, mutação gaussiana) [Holland (1992); Goldberg (1989); Eiben e Smith (2015); Michalewicz (1996); Goldberg e Deb (1991)].
- O tratamento da **aptidão ruidosa** (cada partida depende da sorte do mapa): avaliar todos nos mesmos mapas e reavaliar os melhores [Jin e Branke (2005); Miller e Goldberg (1995); Ross (2013)].
- A separação entre **treino e validação** [Hastie, Tibshirani e Friedman (2009)].

O que foi escolha de projeto, sem fundamentação específica

- Quais características (genes) existem e como cada uma é calculada e normalizada. Na versão atual, as duas principais (risco de morte e distância) foram escolhidas para corresponder aos termos da pontuação, inspiradas na utilidade esperada do AIMA; as demais são intuição.
- Os valores dos parâmetros: taxa e tamanho da mutação, tamanho do torneio, número de elites, limites dos pesos, número de finalistas e tamanho da validação. Eles **não** foram tirados de nenhuma recomendação publicada nem calibrados por um estudo sistemático. Foram escolhidos por julgamento e conferidos só pelo resultado final.
- O genoma inicial que imita o agente probabilístico e o protocolo de pré-treino.

> **SOBRE AS REFERÊNCIAS**
>
> Todas as referências deste documento foram conferidas em buscas na web em 28/10/2026 (títulos, autores, periódicos, volumes e DOIs). O que não pôde ser confirmado foi deixado de fora e está listado no apêndice A. Os DOIs foram confirmados nas páginas das editoras ou em registros bibliográficos; o acesso direto a doi.org estava bloqueado no ambiente usado, então os links não foram abertos um a um.

2 Como o agente funciona



O agente genético herda do agente probabilístico a **base de conhecimento** e duas regras fixas: pegar o ouro quando sente brilho e voltar para a entrada quando já tem o ouro. Só a parte de **exploração** (para onde ir quando ainda não achou o ouro, e se deve atirar) é decidida pela fórmula com pesos evoluídos.

A cada passo, para cada sala da fronteira que não é perigo comprovado:

1. calcula 8 números (as características $f_1 \ldots f_8$);
2. dá uma nota: `nota = w1·f1 + w2·f2 + … + w8·f8`;
3. escolhe a sala de maior nota.

Se ainda tem flecha e o Wumpus está restrito a até 3 salas, calcula também a nota da opção "atirar" com outros 3 pesos. Se ela for maior que a da melhor sala, vai atirar. Os 11 pesos formam o **genoma**, e o algoritmo genético procura os pesos que dão a maior pontuação média.

src/agents/agents.ts — características e nota de cada sala — linhas 304–316

```
  const features = [
    riskOf(b),
    (path.length - 1) / size,
    safe ? 1 : 0,
    neighbors.filter((n) => !kb.visited.has(n)).length / 4,
    (cx + cy) / (2 * size),
    turns,
    neighbors.filter((n) => kb.visited.get(n)?.breeze).length / 4,
    neighbors.filter((n) => kb.visited.get(n)?.stench).length / 4,
  ];
  const score = features.reduce((sum, f, i) => sum + f * g[i], 0);
  if (!best || score > best.score) best = { cell, path, score, safe };
}
```

src/agents/agents.ts — nota da opção de tiro — linhas 319–325

```
if (s.hasArrow && candidates.size > 0 && candidates.size <= 3) {
  const [bias, certainty, safeLeft] = g.slice(CELL_GENES.length);
  const shootScore = bias + certainty * (1 / candidates.size) + safeLeft * (anySafe ? 1 : 0);
  if (!best || shootScore > best.score) {
    const shot = this.planShot(s, candidates, walkable);
    if (shot) return shot;
  }
```

3 Fundamentação de cada componente

3.1 O agente baseado em conhecimento

No jogo: a classe `Knowledge` deduz salas sem poço, possíveis posições do Wumpus e poços certos, e calcula a probabilidade de poço na fronteira por enumeração.

Base: o mundo de Wumpus como problema de agente lógico (cap. 7) e a sua versão probabilística, em que a probabilidade de poço na fronteira é obtida somando sobre as configurações consistentes com as brisas (seção 12.7) [Russell e Norvig (2020)].

Adaptação: a probabilidade de o Wumpus estar numa sala candidata é proporcional à chance de ela não ter poço (o Wumpus é sorteado entre salas sem poço), calculada sala a sala, sem enumerar poços e Wumpus juntos. Essa aproximação é minha.

3.2 Decidir por soma ponderada de características

No jogo: a nota de cada sala é uma combinação linear de características com pesos.

Base: Samuel usou, no programa de damas, um polinômio de avaliação formado por termos (características do tabuleiro) com coeficientes ajustados automaticamente pelo próprio programa [Samuel (1959)]. A ideia de função de avaliação como soma ponderada de características também é apresentada no AIMA, no capítulo sobre jogos [Russell e Norvig (2020)]. Chellapilla e Fogel evoluíram funções de avaliação para damas com computação evolutiva, usando redes neurais em vez de uma soma linear [Chellapilla e Fogel (1999)].

Diferença: aqui a soma é usada para escolher a próxima sala a explorar, não para avaliar posições de um jogo de dois jogadores. Essa transposição é uma adaptação minha.

Características em unidades de utilidade (versão 2, D19). O AIMA define o agente racional como o que escolhe a ação de **maior utilidade esperada** [Russell e Norvig (2020)]. Na pontuação do jogo, entrar numa sala custa, em média, cerca de 1000 × P(morrer lá) pela morte e 1 ponto por passo do caminho. Por isso as duas primeiras características passaram a ser exatamente esses termos: o **risco** P(poço) + P(Wumpus) e a **distância**. Uma soma linear com pesos nessas duas já reproduz a regra do agente probabilístico (menor risco, desempate pela distância), e os outros genes permitem ao treino corrigir o que essa conta não vê, como o valor de abrir área nova. Na versão 1 o genético pesava P(poço) e P(Wumpus) separadamente e precisava acertar a proporção entre os dois pesos para chegar ao mesmo resultado. A escolha das características é escolha de projeto; a ideia de utilidade esperada é da literatura.

3.3 Algoritmo genético

No jogo: `train()` mantém uma população de genomas, avalia, seleciona, cruza e muta por várias gerações.

Base: o algoritmo genético foi formulado por Holland [Holland (1992)] e popularizado por Goldberg [Goldberg (1989)]. O AIMA apresenta algoritmos evolutivos no capítulo 4 [Russell e Norvig (2020)]. Eiben e Smith descrevem o esquema geral (representação, mutação, recombinação, modelo de população, seleção de pais e de sobreviventes) usado como referência de terminologia [Eiben e Smith (2015)].

3.4 Representação com números reais

No jogo: cada gene é um número real entre −5 e 5. A população inicial é sorteada uniformemente nesse intervalo, e os valores mutados são limitados a ele.

Base: representações com valores reais e seus operadores próprios são tratados em [Eiben e Smith (2015); Michalewicz (1996)]. O intervalo ±5 é escolha de projeto.

3.5 Seleção por torneio

No jogo: para escolher cada pai, sorteia 3 genomas e fica com o de maior aptidão (`TOURNAMENT = 3`).

Base: Goldberg e Deb comparam esquemas de seleção, incluindo o torneio [Goldberg e Deb (1991)]. Miller e Goldberg modelam como o tamanho do torneio controla a pressão de seleção e como ela muda quando a aptidão tem ruído, que é o caso deste jogo [Miller e Goldberg (1995)]. O tamanho 3 é escolha de projeto.

3.6 Elitismo

No jogo: os 2 melhores genomas passam sem alteração para a próxima geração (`ELITE = 2`).

Base: preservar os melhores indivíduos entre gerações é uma forma de seleção de sobreviventes, tema tratado em [Eiben e Smith (2015)]. O número 2 é escolha de projeto.

3.7 Cruzamento aritmético

No jogo: cada peso do filho é `pai × t + mãe × (1 - t)`, com *t* sorteado entre 0 e 1 **separadamente para cada peso**.

src/agents/genetic.ts — linhas 65–71

```
function crossover(a: number[], b: number[], rng: () => number) {
  // Cruzamento aritmético por gene: o filho fica num ponto entre os dois pais
  return a.map((v, i) => {
    const t = rng();
    return v * t + b[i] * (1 - t);
  });
}
```

Base: o cruzamento aritmético (combinação convexa dos pais) é atribuído a Michalewicz [Michalewicz (1996)]; implementações como as do HeuristicLab e do Scilab o citam como fonte. Na forma descrita por essas fontes, o mesmo α vale para todos os genes.

Diferença: aqui *t* é sorteado para cada gene, o que é uma adaptação minha. Não verifiquei se essa variante tem um nome estabelecido na literatura, então não atribuo nome a ela.

3.8 Mutação gaussiana

No jogo: cada peso tem 20% de chance de receber um ruído normal de desvio padrão 0,8, e o resultado é limitado a ±5.

src/agents/genetic.ts linhas 48–52

```
function gaussian(rng: () => number) {
  const u = 1 - rng();
  const v = rng();
  return Math.sqrt(-2 * Math.log(u)) * Math.cos(2 * Math.PI * v);
}
```

Base: a mutação por soma de ruído normal é o operador usual para representação real [Eiben e Smith (2015)]. O ruído normal é gerado pela transformação de Box–Muller [Box e Muller (1958)], que é exatamente a fórmula da função `gaussian()` acima. Os valores 20% e 0,8 são escolha de projeto.

3.9 Aptidão ruidosa: os mesmos mapas para todos

No jogo: a aptidão é a pontuação média em M mapas sorteados, e **todos os genomas da mesma geração jogam os mesmos M mapas**. A cada geração, novos mapas são sorteados.

Base: como os mapas são aleatórios, a aptidão é uma medida com ruído. Esse é um dos tipos de incerteza revisados por Jin e Branke, que discutem formas de lidar com ele, como avaliar várias vezes e usar médias [Jin e Branke (2005)]. Usar os mesmos cenários aleatórios para comparar alternativas corresponde à técnica de **números aleatórios comuns**, um método de redução de variância em simulação [Ross (2013)].

Adaptação: a combinação "mesmos mapas dentro da geração, mapas novos entre gerações" é uma adaptação minha dessas ideias. Como todos os genomas da geração jogam os mesmos mapas, comparar as médias é o mesmo que comparar a diferença mapa a mapa: a seleção já é **pareada**.

Quanto ruído sobra. Medido nos mesmos mapas, a diferença de pontuação entre o agente probabilístico e um genoma que o imita tem desvio padrão de cerca de 640 pontos por partida: quando duas estratégias parecidas divergem num mapa, uma pode vencer (+1500) e a outra morrer (−1000). Com M mapas, a incerteza da diferença média é de cerca de 1,96 × 640 ÷ √M: ±56 pontos com 500 mapas, ±28 com 2000 e ±18 com 5000. Por isso o pré-treino da versão 2 usa 500 mapas por geração (antes 120) e a validação final usa 2000 mapas (antes 360). É a **média explícita** discutida por Jin e Branke [Jin e Branke (2005)], com os tamanhos escolhidos por mim a partir dessa conta.

3.10 Validação final em mapas novos

No jogo: na última geração, os 5 melhores jogam 3 × M mapas que nunca viram, e o vencedor dessa rodada é o resultado do treino.

src/agents/genetic.ts linhas 104–113

```
if (gen === opts.generations) {
  // Validação: os finalistas jogam mapas que nunca viram
  const validation = sampleMaps(opts.validationMaps ?? opts.mapsPerGeneration * 3);
  const finalists = scored
    .slice(0, FINALISTS)
    .map((x) => ({ genome: x.genome, fit: fitness(x.genome, validation) }))
    .sort((a, b) => b.fit - a.fit);
  yield { generation: gen, best: finalists[0].fit, average: scored[0].fit, bestGenome:
finalists[0].genome, validated: true };
  return;
}
```

Base: avaliar o modelo escolhido em dados que não foram usados para escolhê-lo, para estimar o desempenho sem o viés da seleção [Hastie, Tibshirani e Friedman (2009)]; e reavaliar soluções promissoras para reduzir o efeito do ruído na aptidão [Jin e Branke (2005)].

Motivo prático: sem essa etapa, o genoma evoluído chegou a ficar **pior** que o escrito à mão (725 contra 760 pontos de média nos mesmos mapas), porque o "melhor" da última geração podia ter tido apenas mapas fáceis. O procedimento específico (5 finalistas, 3 × M mapas na tela, 2000 mapas no pré-treino) é escolha de projeto.

3.11 Treino em vários tamanhos de mapa

No jogo: no pré-treino da versão 2, os mapas de cada geração (e da validação) são divididos entre 4×4 e 6×6, e a aptidão é a média em todos eles.

Base: um modelo ajustado a um único tipo de dado tende a generalizar pior para outros; a avaliação deve usar dados representativos de onde o modelo será usado [Hastie, Tibshirani e Friedman (2009)]. O genoma da versão 1 foi treinado só em 4×4 e já era melhor no 6×6, sinal de que distância e informação pesam mais em mapas grandes; treinar nos dois tamanhos expõe isso à seleção.

Adaptação: a divisão meio a meio entre 4×4 e 6×6 é escolha de projeto.

3.12 Comparação final pareada, com intervalo de confiança

No jogo: `npm run medir` e `npm run treinar` comparam agentes nos mesmos mapas e mostram a diferença média por partida com um intervalo de 95% (média ± 1,96 × desvio ÷ √n, aproximação normal). Um genoma só substitui o padrão se a comparação não indicar perda.

Base: comparar alternativas nos mesmos cenários e analisar a diferença é o uso dos números aleatórios comuns em simulação [Ross (2013)]; a separação entre os mapas de treino, os de seleção e os da medição final segue [Hastie, Tibshirani e Friedman (2009)].

Adaptação: o critério de troca é escolha de projeto.

4 Cada variável do agente

Legenda da coluna "Origem": literatura = conceito tirado da referência indicada; adaptação = conceito da literatura modificado por mim; escolha de projeto = criado por mim, sem fundamentação teórica específica.

4.1 Características de cada sala candidata (genes 1 a 8)

A coluna "Peso padrão" é o valor no genoma que vem com o jogo (`DEFAULT_GENOME`). Peso negativo significa que a característica *diminui* a nota da sala.

GENE	CÁLCULO NO CÓDIGO	FAIXA	O QUE MEDE	ORIGEM	PESO PADRÃO	LEITURA DO PESO
f1 · Risco	`riskOf(b)` = P(poço) + P(Wumpus)	0 a 1	Chance de morrer ao entrar na sala. Poço e Wumpus nunca dividem a sala, então as chances se somam.	adaptação [Russell e Norvig (2020)] (utilidade esperada)	-4.75	Forte rejeição ao risco: é o que mais pesa.
f2 · Distância	`(path.length - 1) / size`	≥ 0	Passos até a sala pelo caminho conhecido, dividido pelo tamanho do mapa. Cada passo custa 1 ponto.	adaptação [Russell e Norvig (2020)] (utilidade esperada)	-0.14	Leve preferência por salas próximas.
f3 · Segura	`safe ? 1 : 0`	0 ou 1	1 se a dedução lógica prova que não há poço nem Wumpus.	literatura [Russell e Norvig (2020)]	1	Preferência extra por salas seguras.
f4 · Novidade	`vizinhas não visitadas / 4`	0 a 1	Quantas vizinhas da sala ainda são desconhecidas.	escolha de projeto	-0.08	Quase neutro.
f5 · Longe da entrada	`(x + y) / (2·size)`	0 a 1	Distância de Manhattan da sala até a entrada, normalizada.	escolha de projeto	-1.44	Prefere salas mais perto da entrada (a volta com o ouro fica mais curta).
f6 · Giros	`0, 0,5 ou 1`	0 a 1	Giro antes do 1º passo: 0 nenhum, 0,5 um giro de 90°, 1 meia-volta.	escolha de projeto	-0.83	Evita girar (cada giro custa 1 ponto).
f7 · Brisas ao lado	`vizinhas visitadas com brisa / 4`	0 a 1	Indício direto de poço por perto.	escolha de projeto	-1.24	Evita salas cercadas de brisa, além do que o risco já mede.
f8 · Fedores ao lado	`vizinhas visitadas com fedor / 4`	0 a 1	Indício direto de Wumpus por perto.	escolha de projeto	-0.41	Leve rejeição a salas com fedor ao lado.

REDUNDÂNCIAS CONHECIDAS

Algumas características se sobrepõem: uma sala "segura" (f3 = 1) sempre tem risco f1 = 0, e as brisas e fedores ao lado (f7, f8) já influenciam o risco. Elas foram mantidas porque permitem ao algoritmo genético pesar o mesmo tipo de informação de formas diferentes, mas isso não foi testado contra uma versão sem redundância.

4.2 Características da opção de tiro (genes 9 a 11)

A opção só é considerada se o agente tem a flecha e o Wumpus está restrito a no máximo 3 salas (o limite 3 é escolha de projeto). A nota é `s1 + s2·(1 ÷ candidatas) + s3·(há sala segura)` e é comparada com a nota da melhor sala.

GENE	CÁLCULO NO CÓDIGO	FAIXA	O QUE MEDE	ORIGEM	PESO PADRÃO	LEITURA DO PESO
s1 · Tiro: viés	`1`	1	Termo constante: vontade de atirar independente do resto.	escolha de projeto	-1.1	Resistência a gastar a flecha.
s2 · Tiro: certeza	`1 / candidatas`	1/3 a 1	Chance de o alvo ser o Wumpus (1 se ele foi localizado).	adaptação [Russell e Norvig (2020)]	1.58	Atira mais quando tem certeza.
s3 · Tiro: há sala segura	`anySafe ? 1 : 0`	0 ou 1	1 se ainda existe sala segura para explorar.	escolha de projeto	-3.19	Não gasta a flecha enquanto houver caminho seguro.

4.3 Parâmetros do algoritmo genético

VARIÁVEL	VALOR	PAPEL	ORIGEM
`GENE_LIMIT`	5	Cada peso fica entre −5 e 5 (inicialização e mutação).	escolha de projeto; representação real [Eiben e Smith (2015)]
`population`	20 na tela e no pré-treino	Quantos genomas existem em cada geração.	escolha de projeto
`generations`	15 na tela; 20 no pré-treino	Quantas gerações o treino dura.	escolha de projeto
`mapsPerGeneration`	20 na tela; 500 no pré-treino	Quantos mapas cada genoma joga por geração (o M do texto).	escolha de projeto; ver 3.9 [Jin e Branke (2005)]
`sizes`	tamanho atual na tela; 4×4 e 6×6 no pré-treino	Tamanhos dos mapas de treino, divididos igualmente.	escolha de projeto; ver 3.11 [Hastie, Tibshirani e Friedman (2009)]
`randomGenome()`	uniforme em [−5, 5]	Como os genomas iniciais são sorteados.	literatura [Eiben e Smith (2015)]
`seed`	genoma atual	O genoma em uso entra na posição 0 da população inicial, para o treino continuar de onde parou.	escolha de projeto
`TOURNAMENT`	3	Tamanho do torneio de seleção dos pais.	literatura [Goldberg e Deb (1991); Miller e Goldberg (1995)]; valor: escolha de projeto
`ELITE`	2	Quantos melhores passam sem alteração.	literatura [Eiben e Smith (2015)]; valor: escolha de projeto
`crossover()`	t ~ U(0, 1) por gene	Cruzamento aritmético.	adaptação [Michalewicz (1996)]
`MUTATION_RATE`	0,2	Chance de cada peso sofrer mutação.	literatura [Eiben e Smith (2015)]; valor: escolha de projeto

VARIÁVEL	VALOR	PAPEL	ORIGEM
`MUTATION_SIZE`	0,8	Desvio padrão do ruído da mutação.	literatura [Eiben e Smith (2015); Box e Muller (1958)]; valor: escolha de projeto
`FINALISTS`	5	Quantos melhores vão para a validação final.	adaptação [Jin e Branke (2005); Hastie, Tibshirani e Friedman (2009)]; valor: escolha de projeto
`validationMaps`	3 × M na tela; 2000 no pré-treino	Tamanho da rodada de validação final.	escolha de projeto ; ver 3.9
`layout`	opcional	"Treinar só no mapa atual": todos os genomas jogam um único mapa fixo. O agente se especializa nele e deixa de generalizar.	escolha de projeto
`rng`	`Math.random`	Fonte dos sorteios do treino (mapas, pais, mutação).	—

4.4 A aptidão (fitness)

A aptidão é a **pontuação média** do genoma nos mapas da geração. A pontuação de cada partida segue a medida de desempenho do mundo de Wumpus do AIMA [Russell e Norvig (2020)]: −1 por ação, −10 pelo tiro, −1000 por morrer e +1000 pelo ouro.

No livro, os +1000 são ganhos ao sair da caverna com o ouro. Neste jogo eles foram divididos em +1000 ao pegar o ouro e +500 ao voltar com ele à entrada, foi criado um limite de ações por partida e matar o Wumpus passou a valer +25 (pequeno o bastante para que atirar às cegas continue dando prejuízo em média). Essas quatro mudanças são escolha de projeto e alteram a aptidão em relação à medida original.

4.5 Os genomas prontos

GENE	1	2	3	4	5	6	7	8	9	10	11
`BAYES_LIKE_GENOME`	-5	-0.01	0	0	0	0	0	0	0	2	-5
`DEFAULT_GENOME`	-4.75	-0.14	1	-0.08	-1.44	-0.83	-1.24	-0.41	-1.1	1.58	-3.19

- **`BAYES_LIKE_GENOME`** (escolha de projeto): escrito à mão para reproduzir o agente probabilístico. Risco −5 (decide), distância −0,01 (só desempata), demais características em 0, e atira só quando não há sala segura (−5 em s3) e com certeza suficiente (+2 em s2). Serve de ponto de partida do treino.
- **`DEFAULT_GENOME`** (versão 2): resultado de um treino a partir do `BAYES_LIKE_GENOME` em mapas 4×4 e 6×6 com 20% de poços, com população 20, 20 gerações, 500 mapas por geração (metade de cada tamanho) e validação final em 2000 mapas novos. Foram feitos três treinos (sementes `utilidade-a` , `-b` e `-c`); o escolhido foi o melhor em 4000 mapas 4×4 e 800 mapas 6×6 de seleção, separados dos mapas da medição final (seção 5). Para reproduzir: `npm run treinar -- --seed utilidade-a --pop 20 --geracoes 20 --mapas 500 --tamanhos 4,6 --validacao 2000` .

Os pesos evoluídos mantiveram o risco como critério principal (−4,75) e a cautela para atirar havendo sala segura (−3,19). O treino acrescentou três preferências que a regra do probabilístico não tem: ficar **perto da entrada** (−1,44, encurta a volta com o ouro), **evitar giros** (−0,83) e **evitar salas cercadas de brisa** (−1,24) além do que o risco já mede. São justamente correções de custo e de cautela que fazem mais diferença em mapas grandes.

4.6 O que não é evoluído

- Pegar o ouro quando sente brilho e voltar à entrada pelo caminho conhecido quando já tem o ouro (regras fixas herdadas do agente probabilístico).
- Excluir da escolha as salas que são perigo comprovado (poço certo ou Wumpus localizado).
- Como mirar e atirar depois de decidir atirar (rotina `planShot`).
- As próprias deduções lógicas e probabilidades: o algoritmo genético só decide quanto cada informação pesa, não como ela é calculada.

5 Resultados obtidos e o que eles significam

Estes números são experimentos feitos durante o desenvolvimento, não resultados da literatura. Todos os agentes jogaram exatamente os mesmos mapas, com 20% de poços (3 no 4×4, 7 no 6×6), sorteados pelas sementes `medicao-4x4` e `medicao-6x6`, que não foram usadas nem no treino nem na escolha do genoma.

AGENTE	4×4 (5000 MAPAS)	6×6 (1000 MAPAS)
Lógico	63% de vitórias, 570 pts	43%, 46 pts
Probabilístico	67%, 650 pts	48%, 182 pts
`BAYES_LIKE_GENOME` (ponto de partida)	66%, 642 pts	49%, 188 pts
`DEFAULT_GENOME` (evoluído, versão 2)	66%, 638 pts	51%, 242 pts

Comparação pareada com o agente probabilístico

	4×4	6×6
Diferença média por partida (genético − probabilístico)	−12 pts	+60 pts
Intervalo de 95%	−49 a +25	−18 a +138
Conclusão	sem diferença comprovada	sem diferença comprovada, com tendência a favor do genético

- O `BAYES_LIKE_GENOME`, só com risco e distância, joga praticamente igual ao agente probabilístico nos dois tamanhos (diferença de −7 e +6 pts, ambas dentro do ruído). Isso confirma a ideia da seção 3.2: com as características em unidades de utilidade, a regra do probabilístico é um caso particular da fórmula.
- O genoma evoluído empata no 4×4 e tende a ganhar no 6×6. A vantagem em mapas maiores é coerente com os pesos aprendidos (perto da entrada, poucos giros), que só importam quando os caminhos são longos.
- Mesmo com 5000 mapas, a margem de erro da diferença é de cerca de ±37 pts no 4×4: os dois agentes divergem raramente, mas quando divergem um pode vencer e o outro morrer. Ganhos menores que isso não podem ser comprovados com essa quantidade de mapas.
- A versão 1 do genoma (características separadas de poço e Wumpus, treino só em 4×4 com 120 mapas por geração) tinha números parecidos (637 e 255 pts), mas sem compromisso com o alvo: pesos treinados na tela podiam fazê-lo andar em círculos até o tempo esgotar (53% de tempo esgotado numa Arena 6×6). A versão 2 corrige isso e torna o treino reproduzível e medido com intervalo de confiança.
- O bônus de +25 por matar o Wumpus não fez o agente atirar às cegas: a fração de partidas com tiro é praticamente a mesma do probabilístico (39% no 4×4, 33% no 6×6).

6 Limitações da fundamentação

- **Sem calibração de parâmetros.** Nenhum parâmetro (taxa e tamanho da mutação, torneio, elites, população, gerações, mapas, limites dos pesos) foi escolhido por um estudo de sensibilidade. Com outros valores o resultado pode ser melhor ou pior.
- **Características escolhidas à mão.** O conjunto de genes reflete intuições sobre o problema; não segue nenhum método publicado de seleção de características.
- **Análise estatística simples.** A comparação do capítulo 5 usa diferença pareada com intervalo de 95% pela aproximação normal; não houve correção para várias comparações nem outros testes.
- **Mudanças na medida de desempenho.** A aptidão segue o AIMA com três alterações (divisão do bônus, vitória automática, limite de ações), então os números não são diretamente comparáveis com outros trabalhos sobre o mundo de Wumpus.
- **Modelo do Wumpus simplificado.** A probabilidade do Wumpus usa a chance de poço de cada sala separadamente e ignora a correlação entre as salas da fronteira.

7 Referências

1. RUSSELL, Stuart; NORVIG, Peter. *Artificial Intelligence: A Modern Approach*. 4. ed. Hoboken: Pearson, 2020. ISBN 978-0-13-461099-3. Site dos autores: aima.cs.berkeley.edu. Página da editora: pearson.com. Capítulos usados: cap. 4 (*Search in Complex Environments*, que inclui algoritmos evolutivos), cap. 7 (*Logical Agents*, mundo de Wumpus lógico) e seção 12.7 (*The Wumpus World Revisited*, Wumpus probabilístico).

2. SAMUEL, Arthur L. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, v. 3, n. 3, p. 210–229, 1959. DOI: 10.1147/rd.33.0210. Cópia em PDF: people.csail.mit.edu/brooks/idocs/Samuel.pdf.

3. CHELLAPILLA, Kumar; FOGEL, David B. Evolving neural networks to play checkers without relying on expert knowledge. *IEEE Transactions on Neural Networks*, v. 10, n. 6, p. 1382–1391, 1999. DOI: 10.1109/72.809083.

4. HOLLAND, John H. *Adaptation in Natural and Artificial Systems*. Cambridge: MIT Press, 1992. ISBN 978-0-262-58111-0. (Primeira edição pela MIT Press; a obra foi publicada originalmente em 1975.)

5. GOLDBERG, David E. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading: Addison-Wesley, 1989. ISBN 0-201-15767-5.

6. EIBEN, A. E.; SMITH, J. E. *Introduction to Evolutionary Computing*. 2. ed. Berlin: Springer, 2015. (Natural Computing Series). DOI: 10.1007/978-3-662-44874-8.

7. MICHALEWICZ, Zbigniew. *Genetic Algorithms + Data Structures = Evolution Programs*. 3. ed. rev. e ampl. Berlin: Springer, 1996. DOI: 10.1007/978-3-662-03315-9.

8. GOLDBERG, David E.; DEB, Kalyanmoy. A comparative analysis of selection schemes used in genetic algorithms. In: RAWLINS, G. J. E. (ed.). *Foundations of Genetic Algorithms*, v. 1, p. 69–93, 1991. Cópia em PDF: cse.unr.edu/~sushil/class/gas/papers/Select.pdf.

9. MILLER, Brad L.; GOLDBERG, David E. Genetic algorithms, tournament selection, and the effects of noise. *Complex Systems*, v. 9, n. 3, 1995. Resumo: complex-systems.com/abstracts/v09_i03_a02.

10. JIN, Yaochu; BRANKE, Jürgen. Evolutionary optimization in uncertain environments — a survey. *IEEE Transactions on Evolutionary Computation*, v. 9, n. 3, p. 303–317, 2005. DOI: 10.1109/TEVC.2005.846356. Cópia em PDF: cs.le.ac.uk/people/sy11/ECiDUE/JinBranke-TEVC05.pdf.

11. ROSS, Sheldon M. *Simulation*. 5. ed. Amsterdam: Academic Press (Elsevier), 2013. ISBN 978-0-12-415825-2. (Capítulo de técnicas de redução de variância, que inclui o uso de números aleatórios comuns.)

12. HASTIE, Trevor; TIBSHIRANI, Robert; FRIEDMAN, Jerome. *The Elements of Statistical Learning*. 2. ed. New York: Springer, 2009. PDF hospedado pelo autor: web.stanford.edu/~hastie/Papers/ESLII.pdf. (Cap. 7, *Model Assessment and Selection*.)

13. BOX, G. E. P.; MULLER, Mervin E. A note on the generation of random normal deviates. *The Annals of Mathematical Statistics*, v. 29, n. 2, p. 610–611, 1958. DOI: 10.1214/aoms/1177706645. Acesso aberto: projecteuclid.org/euclid.aoms/1177706645.

A Apêndice: o que não foi possível verificar

Estes itens foram considerados mas ficaram **fora** das referências, porque não consegui confirmar nas fontes consultadas:

- **Números das subseções do AIMA (4ª edição)** para algoritmos evolutivos e funções de avaliação. Os títulos dos capítulos 4 e 7 e a seção 12.7 foram confirmados; as subseções não, por isso o texto cita só os capítulos.
- **Atribuição do elitismo a De Jong (1975).** A tese de De Jong existe (*An analysis of the behavior of a class of genetic adaptive systems*, University of Michigan, 1975), mas as fontes consultadas não confirmaram que ela introduz o elitismo. O elitismo é citado aqui apenas por meio de Eiben e Smith.
- **Livro de simulação de Law.** A seção sobre números aleatórios comuns foi confirmada apenas para a 4ª edição, por um programa de curso. Em seu lugar foi usado Ross (2013).
- **Definição literal do cruzamento aritmético em Michalewicz.** A atribuição foi confirmada por fontes secundárias (documentação do HeuristicLab e do Scilab), não pela leitura do trecho do livro.
- **Números de página de Miller e Goldberg (1995).** Volume e número foram confirmados; as páginas não, por isso foram omitidas.